

Code Challenge: Building an AI-Powered Candidate Screening and Scoring System

Prompt:

Build an AI-powered (Machine Learning) application that can effectively screen and score candidates for a specific role based on their resumes and other relevant information. The system should leverage natural language processing (NLP) techniques to extract key information from resumes and apply scoring criteria to rank candidates.

Input

We will provide:

- A candidate database
- 2 Job descriptions for your tests

Submission Guidelines:

- **Deadline:** 72 hours after submission of this prompt
- **Code Repository:** Please submit a link to a public GitHub repository containing your project code, including a list of library dependencies.
- **Project Report:** A concise report detailing the project's design, implementation, and challenges faced. (max 1 page)
- **Demo:** A brief video demonstration showcasing your code challenge solution. Max 5 minutes. Upload it to <https://www.awesomescreenshot.com/> or <https://www.loom.com/>

Technical Requirements:

1. Data Ingestion and Preprocessing:
 - Clean and preprocess the database to remove noise and inconsistencies.
 - Tokenize and vectorize text data (e.g., resumes, job descriptions) using appropriate NLP techniques.

2. Feature Extraction:

- Extract relevant features from the candidate data, such as experience, skills, education, and certifications.
- Consider using techniques like TF-IDF and word embeddings to represent text data.

3. Model Development:

- Train a machine learning model (e.g., classification, regression) to predict candidate scores based on the extracted features.
- Experiment with different algorithms (e.g., decision trees, random forests, neural networks) and hyperparameters to optimize performance.
- Consider using techniques like cross-validation to assess model generalization.

4. Web Application Development:

- Build a user-friendly web application using HTML and Python to allow recruiters to upload job descriptions and view the scoring results.
 - Optional use of Javascript is allowed
- Web page interface needs to:
 - Allow recruiters to define a scoring criteria
 1. Should be a Text area to input a paragraph with the desired job description to score. Max 200 characters.
 - Show results with top candidate scores and rankings
 1. Top 30 candidates with their scores.
 2. Max score: 100 points
- Feel free to use any Python framework or libraries (e.g., Flask, FastAPI) to handle data processing, and connect with your Machine Learning model
- Styles will not be evaluated, so please don't spend time adding styles
- **Please don't add user authentication to this website.**
- Considering the time limit, We highly recommend to **pick a simple solution for build the website**

Example:

Your solution should include an intuitive interface where the user can input a Job description to score candidates in the database and show the top 30 best candidates for that role. Role description example: “Senior software Ruby engineer with PostgreSQL experience”

Evaluation Criteria:

- **Code Quality:**
 - Clean, well-structured, and commented code.
 - Adherence to best practices for Python and JavaScript programming.
- **Model Performance:**
 - Accurate prediction of candidate scores based on the defined criteria.
- **Application Functionality:**
 - User-friendly interface for data upload and result visualization.
 - Efficient and reliable backend API.
 - Scalable deployment.
- **Problem-Solving and Innovation:**
 - Creative approaches to data preprocessing, feature extraction, and model selection.
 - Ability to identify and address potential challenges.
- **BONUS (extra points):**
 - Deploy the application to a cloud platform (e.g., AWS, GCP, Azure) for accessibility.